Characterizing Lazy Deletion Overhead in Dijkstra's Algorithm: A Data-Structure Selection Framework Based on Average Degree

Gwanwoo Park
Algorithm Research Lab

***Abstract*—Dijkstra's algorithm with a lazy deletion heap (heapq) is the de facto standard for shortest-path computation, yet the overhead of its "push-and-skip" strategy has not, to our knowledge, been systematically quantified as a closed-form empirical model of graph structure. We identify *skip_ratio*—the fraction of heap pops discarded as stale—as the primary hidden cost, and show empirically that it follows a power law in average degree k: skip_ratio$(k) \approx 1 - 0.884/k^{0.262}$ (R^2=0.984, $\mathrm{U}[1, 100]$ weights, OLS SE ± 0.013 on the exponent), independent of graph size V across more than three orders of magnitude ($V \in [500,\ 10^6]$; source-vertex CV $\leq 2.5\%$). Systematic experiments across weight distributions reveal an empirical *two-regime model*: the power-law exponent α is strongly associated with $\log(W_{\max}/W_{\min})$ and the fitted k-range, and the local slope converges toward $1/H_k$ as $k \to \infty$—connecting the empirical formula to the classical harmonic record-minimum bound. We further show that naïve push-blocking is fundamentally flawed (up to 74% incorrect distances), while Dial's bucket queue achieves up to $4.3\times$ peak speedup (geometric mean $2.9\times$ in C++, $1.5\times$ in Python) via a "natural pruning" mechanism, and Fibonacci heap is 1.5–$2.3\times$ *slower* than heapq despite its asymptotic optimality. The power-law formula generalizes to Barabási–Albert scale-free graphs ($< 8\%$ error) but diverges on planar road networks—consistent with the locally tree-like structure underlying the formula's modeling assumptions. We conclude with a practitioner-ready data-structure selection framework grounded in k and weight type.**

***Index Terms*—Dijkstra's algorithm, lazy deletion, priority queue, Fibonacci heap, Dial's algorithm, skip ratio, graph algorithms, shortest path**

I. Introduction

Dijkstra's algorithm [1] remains the workhorse of single-source shortest-path computation, deployed in routing engines, transportation planning, social-network analysis, and game-AI pathfinding. In virtually every practical implementation the underlying priority queue is a *lazy deletion heap*: whenever a better path to vertex v is found, a new entry (d', v) is pushed onto the heap without removing the now-stale entry (d, v) already there. Stale entries are detected and discarded silently when they surface at the heap minimum. This strategy requires no decrease-key operation, simplifying both implementation and cache behavior, and is recommended in every major algorithms textbook and competitive-programming guide.

Yet the cost of these silently discarded pops—what we term **skip_ratio**—has not, to the best of our knowledge, been systematically quantified as a closed-form empirical model of graph structure. Prior experimental surveys (e.g., Cherkassky et al. [7], Larkin et al. [8]) treat the lazy deletion overhead as an implementation detail rather than decomposing it by graph properties. As a result, no principled criterion exists for deciding *when* the overhead is worth replacing with an alternative data structure.

This paper addresses three research questions:

1) **RQ1.** What graph property determines skip_ratio, and can it be captured by a closed-form empirical model?
2) **RQ2.** Can simple algorithmic modifications (e.g., capping heap duplicates) reduce the overhead without sacrificing correctness?
3) **RQ3.** Which alternative data structures genuinely eliminate the overhead, and under what conditions do they deliver practical speedup?

Contributions. This work makes five concrete contributions:

- We propose and empirically validate a power-law formula skip_ratio$(k) \approx 1 - c/k^{\alpha}$ (c=0.884, α=0.262, R^2=0.984) that is *independent of* V for $V \in [500,\ 10^6]$ (Experiments 2 and A).
- We characterize the **weight-distribution dependence** of α: it is strongly associated with $\log(W_{\max}/W_{\min})$ and grows from 0 (constant weights) to ≈0.32 (continuous uniform). For large k, the local slope converges toward $1/H_k$, revealing a conjectured *two-regime model* (Equation (2)) that connects the power law to the classical harmonic record-minimum bound.
- We demonstrate that threshold-based push blocking is structurally broken: it creates propagation gaps that yield up to 74% incorrect distances with no speedup benefit—a negative result that reveals an inherent property of the lazy deletion data flow.
- We characterize Dial's "natural pruning" mechanism—up to 49% of pushes discarded without processing—as the root cause of its **up-to-$4.3\times$ peak speedup** over heapq in C++ (geometric mean $2.9\times$ in C++, $1.5\times$ in Python).
- We present a practitioner-ready **data-structure selection framework** grounded in k and weight type, validated in both Python and C++ and on three real-world SNAP

datasets.

Taken together, these contributions answer the following unifying claim: *We empirically characterize the hidden overhead of lazy deletion Dijkstra as a function of average degree k, and propose a data-structure selection framework that achieves up to $4.3\times$ peak speedup (geometric mean $2.9\times$ in C++) by matching graph structure to the appropriate priority queue.*

The remainder of the paper is organized as follows. Section II reviews related work. Section III reviews the lazy deletion mechanism and alternative structures. Section IV presents our empirical characterization of skip_ratio. Section V shows why naïve fixes fail. Section VI compares the three data structures. Section VII validates the formula on real datasets. Section VIII discusses limitations and future work. Section IX concludes.

II. Related Work

The complexity of Dijkstra's algorithm has been studied extensively. Fredman and Tarjan [2] established the $O(V \log V + E)$ bound using Fibonacci heaps. Dial [3] introduced the bucket queue for integer-weight graphs, achieving $O(V + E + W)$. Goldberg et al. [4] survey practical shortest-path implementations and find that binary-heap variants outperform Fibonacci heaps on road networks due to cache effects. Sanders and Schultes [5] achieve sub-millisecond continental routing via contraction hierarchies, but their preprocessing is graph-specific.

Cherkassky et al. [7] provide the most comprehensive experimental comparison of shortest-path priority queues to date, establishing binary heaps as the practical baseline across a broad range of graph families. Larkin et al. [8] conduct a back-to-basics empirical study of priority queues and confirm that simple binary heaps dominate in practice—a finding our work extends by quantifying *why*: the hidden lazy deletion overhead, expressed as a function of average degree, is the key differentiator. However, neither study decomposes the lazy deletion overhead as a function of average degree, nor proposes a closed-form empirical model for the skip-pop fraction.

Beyond binary heaps, d-ary heaps and pairing heaps have been studied as alternatives [13]. D-ary heaps reduce the height of the heap tree, lowering the decrease-key cost at the expense of more comparisons per extract-min. Pairing heaps achieve amortized $O(\log n)$ decrease-key with simpler structure than Fibonacci heaps. Crucially, *all* of these structures admit a lazy deletion variant; our skip_ratio metric therefore applies uniformly whenever decrease-key is omitted in favor of push-and-skip. The DIMACS shortest-path implementation challenge [14] provides standardized benchmark graphs and motivates the real-world validation in Section VII.

Our work complements this literature by *quantifying the lazy deletion overhead itself*—a factor that prior engineering studies treat as given rather than as a variable to optimize. Closest in spirit is Chen et al. [6], who study priority queue performance in shortest-path algorithms, but without decomposing overhead by average degree or deriving a predictive formula. To the best of our knowledge, no prior work proposes a closed-form empirical power-law model $\text{skip\_ratio}(k) \approx 1 - c/k^{\alpha}$ or a data-structure selection framework grounded in skip_ratio as a function of average degree.

III. Background

A. Lazy Deletion Dijkstra

Algorithm 1 presents the lazy deletion variant we study. The key difference from the textbook version is line 7: instead of performing a decrease-key on the existing heap entry for vertex v, a fresh entry with the improved distance is simply pushed. When an entry (d, u) is popped (line 4), it is skipped if $d > \text{dist}[u]$ (line 5), meaning a better path has since been found.

Algorithm 1 Dijkstra with Lazy Deletion Heap

```
1:  dist[s] ← 0;  push (0, s)
2:  while heap not empty do
3:      (d, u) ← ExtractMin()
4:      if d > dist[u] then
5:          skip {stale entry}
6:      end if
7:      for each (v, w) ∈ adj[u] do
8:          if d + w < dist[v] then
9:              dist[v] ← d + w;  push (d + w, v)
10:         end if
11:     end for
12: end while
```

We define:

$$\text{skip\_ratio} = \frac{\text{skip\_pop}}{\text{total\_pop}} = 1 - \frac{V}{\text{push\_count}},$$

where the second equality holds for the **lazy binary heap** as follows: every non-skip pop settles exactly one vertex, so total_pop − skip_pop $= V$; in the standard implementation the while-loop runs until the heap is empty, so total_pop = push_count exactly, giving skip_ratio $= 1 - V/\text{push\_count}$. For **Dial's bucket queue**, some pushed entries are abandoned in future buckets without ever being popped (Section VI-B), so push_count > total_pop; in that case we measure skip_ratio directly as skip_pop/total_pop from instrumented bucket operations.

B. Dial's Bucket Queue

Dial's algorithm [3] replaces the heap with a circular array of $W + 1$ buckets, where W is the maximum edge weight. Each bucket b holds vertices whose current tentative distance maps to $b \bmod (W + 1)$. Stale entries are detected lazily: when processing bucket b at distance d, vertex u is skipped if $\text{dist}[u] \neq d$. The time complexity is $O(V + E + W)$.

C. Fibonacci Heap

The Fibonacci heap [2] supports decrease-key in $O(1)$ amortized time, enabling a single heap entry per vertex. As a result, stale entries never arise: every pop is a genuine

vertex settlement. The total complexity is $O(V \log V + E)$. In practice, however, the pointer-chasing structure incurs high constant factors and cache-miss rates.

IV. Empirical Analysis of skip_ratio

A. Theoretical Motivation

Consider a vertex v with degree k in a random graph. As Dijkstra progresses, v's neighbors settle one by one and attempt to relax v. Under the *locally tree-like assumption*—that the r-hop neighborhood of any vertex is a tree with high probability, as in Erdős–Rényi graphs [9]—the order in which neighbors settle is approximately uniform.

By a standard record-low argument [11]: if k independent relaxation attempts arrive in uniform random order, the expected number that strictly improve the current tentative distance is the harmonic number

$$\mathbb{E}[\text{pushes to } v] \;=\; H_k \;=\; \sum_{i=1}^{k} \frac{1}{i} \;\approx\; \ln k + \gamma,$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant. Each subsequent relaxation that improves the distance generates a push, making the previous entry stale. Hence push_per_node $\approx H_k$, and

$$\text{skip\_ratio}(k) \;\approx\; 1 - \frac{1}{H_k} \;\approx\; 1 - \frac{1}{\ln k + \gamma}.$$

The harmonic formula $1 - 1/H_k$ captures the *qualitative* trend (skip_ratio increases with k, independently of V), but it functions as a theoretical *upper bound* on skip_ratio, not a quantitative predictor: it overestimates measured skip_ratio by 12%–44% across $k \in [4, 512]$ for $\mathrm{U}[1, 100]$ weights, and underestimates it at very small k. The discrepancy arises because the i.i.d. settling-order assumption breaks down under correlated edge weights and finite degree heterogeneity—conditions present in every real Dijkstra execution.

Our experiments across $k \in [4, 512]$ show that a power-law model provides a materially tighter empirical fit, with root-mean-square residual more than $4\times$ smaller than the harmonic formula:

$$\boxed{\text{skip\_ratio}(k) \;\approx\; 1 - \frac{c}{k^{\alpha}}, \quad c = 0.884,\ \alpha = 0.262} \qquad (1)$$

with $R^2 = 0.984$ for uniform integer weights on $[1, 100]$. While V-independence is implied qualitatively by the harmonic theory, our formula quantifies it precisely and demonstrates that a power law provides the tightest empirical fit ($4\times$ smaller RMS residual than the harmonic formula) in the practically relevant range $k \in [4, 512]$. The exponent α is not a universal constant: we show in Section IV-D that it depends on the weight distribution through the log-ratio $\log(W_{\max}/W_{\min})$, converging to $\alpha_\infty \approx 0.309$ as the weight range grows large.

Remark (Scope of validity). The locally tree-like assumption holds well for Erdős–Rényi-like random graphs but is violated by structured graphs (planar, hierarchical). We return to this in Sections VII and VIII.

B. Experimental Setup

Graph generation. We generate undirected random graphs with a fixed target average degree k by sampling $\lfloor Vk/2 \rfloor$ random edges uniformly over all vertex pairs and appending a random spanning path for connectivity. This approximates the $G(n, m)$ Erdős–Rényi model with $m = \lfloor Vk/2 \rfloor$; multi-edges are possible but occur with probability $O(k/V)$ and are negligible for $V \geq 1{,}000$. The spanning path ensures connectivity without materially altering degree statistics; the resulting average clustering coefficient is $O(k/V)$, confirming the locally tree-like regime required by the theory in Section IV. Edge weights are drawn uniformly from $[1, 100]$ (integer) for Sections IV–VI and $[0, 1]$ (float) for Section VI.

Parameters.

- $V \in \{500,\ 1{,}000,\ 5{,}000,\ 10{,}000,\ 50{,}000,\ 100{,}000\}$
- $k \in \{4,\ 8,\ 16,\ 32,\ 64,\ 128,\ 256,\ 512\}$
- Source vertex: node 0 (fixed seed 42)
- Repeats: 5–10 runs; mean reported

Hardware. All experiments run on Google Cloud Platform `e2-standard-4` (4 vCPU, 16 GB RAM, Debian 12) unless otherwise noted. C++ binaries compiled with `g++ -O2 -std=c++17`. Python 3.11, NumPy 2.4.

Reproducibility. Code, experiment scripts (Python and C++), and raw CSV data are available at https://github.com/Korean3247/dijkstra-skip-ratio.

C. Results

Fitting procedure. Let $\bar{\text{sr}}_i$ denote mean skip_ratio at degree k_i, averaged across all tested V values. We set $Y_i = \log(1 - \bar{\text{sr}}_i)$ and $X_i = \log k_i$, and apply ordinary least squares (OLS) to the linear model $Y_i = \log c - \alpha X_i$. The fit uses $k_i \in \{4, 8, 16, 32, 64, 128, 256, 512\}$ pooled from Experiments 2 and A (means across all V values at each k), yielding $\hat{c} = 0.884$, $\hat{\alpha} = 0.262$, $R^2 = 0.984$. OLS standard errors, computed from the residual variance on $n = 8$ support points, are $\mathrm{SE}(\hat{\alpha}) \approx 0.013$ and $\mathrm{SE}(\hat{c}) \approx 0.032$. For the practical range $k \in [4, 64]$, a complementary fit on the theory-probe dataset (Section IV-D) gives $\hat{\alpha} = 0.293 \pm 0.012$ (OLS SE) with bootstrap 95% CI $[0.254,\ 0.323]$ (B=10,000). The higher value relative to the full-range estimate ($\hat{\alpha} = 0.262$, $k \in [4, 512]$) is expected: as the fitted k-range expands, $\hat{\alpha}$ decreases because the power-law exponent is itself a function of k in the two-regime model (Finding 1, Section IV-D). Both estimates are valid within their respective k-ranges. The same log-log OLS procedure is applied independently for each weight distribution in Section IV-D.

Power-law fit. Figure 1 plots skip_ratio against k for $V = 1{,}000$ and $V = 10{,}000$ alongside the fitted curve (1). Residual magnitudes vary systematically with k: the formula fits best near $k = 8$ (residual $<$0.001); for $k \in [16, 64]$ it *under*-predicts measured skip_ratio by up to 0.022 (at $k = 32$); for $k \geq 128$ it *over*-predicts as the true curve bends toward the harmonic regime, reaching $+0.051$ at $k = 512$. For $k = 4$ the formula over-predicts by 0.021, attributable to the locally tree-like assumption being weakest at the lowest degrees. These

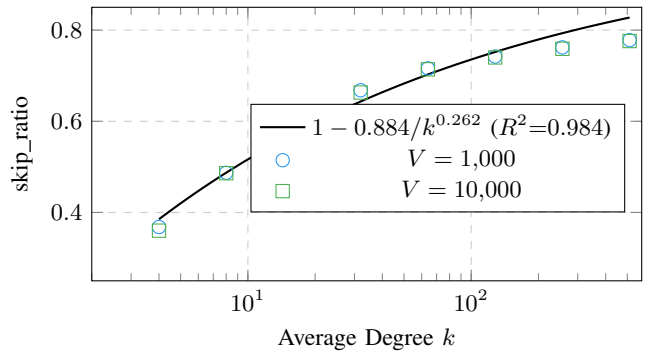

Fig. 1. skip_ratio vs. average degree k (log scale on x-axis). Circles: $V = 1{,}000$; squares: $V = 10{,}000$. The fitted power-law curve (1) is plotted in black. Data pooled from Experiments 2 and A ($k \in [4, 512]$).

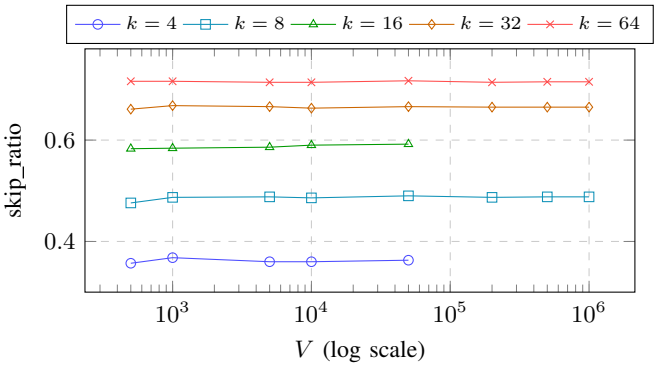

Fig. 2. skip_ratio vs. V for values of k (Experiments 2 and A). Filled symbols: Exp. 2 ($V \leq 50{,}000$); open-circle continuation: Exp. A ($V \in \{200K, 500K, 1M\}$). All series are flat from $V = 500$ to $V = 10^6$: skip_ratio does not depend on graph size across more than three orders of magnitude ($V \in [500, 10^6]$).

TABLE I

SKIP_RATIO ACROSS V AT FIXED k (EXPERIMENTS 2 AND A). VARIATION WITHIN EACH COLUMN IS ≤ 0.013 ACROSS MORE THAN THREE ORDERS OF MAGNITUDE ($V \in [500, 10^6]$). TABLE SHOWS REPRESENTATIVE ROWS ONLY; INTERMEDIATE VALUES ($V \in \{5{,}000, 10{,}000, 100{,}000\}$ FROM EXP. 2, PLOTTED IN FIGURE 2) LIE WITHIN THE SAME RANGE. ENTRIES MARKED "—" WERE NOT MEASURED AT THAT (V, k) COMBINATION.

V	Exp.	Average degree k				
		4	8	16	32	64
500	2	0.357	0.476	0.583	0.661	0.716
1,000	2	0.368	0.487	0.584	0.668	0.716
50,000	2	0.363	0.490	0.592	0.666	0.717
200,000	A	—	0.487	—	0.665	0.714
500,000	A	—	0.488	—	0.665	0.715
1,000,000	A	—	0.488	—	0.665	0.715

signed residuals are consistent with the two-regime model (Section IV-D): the fitted power law makes the optimal trade-off across the full range but has non-negligible bias at the extremes.

V-independence. Table I reports skip_ratio across more than three orders of magnitude in V ($V \in [500, 10^6]$, ratio 2,000×) for representative values $k \in \{4, 8, 16, 32, 64\}$, extended to $V \in \{200{,}000; 500{,}000; 1{,}000{,}000\}$ in Experiment A. The maximum variation at fixed k is $\Delta \leq 0.013$ across the full range, confirming that skip_ratio is determined primarily by k (for fixed weight distribution and graph model). Figure 2 visualizes this: the curves for each k are essentially flat from $V = 500$ to $V = 10^6$.

D. Weight Distribution Dependence

Equation (1) was fitted for uniform integer weights on $[1, 100]$. A natural question is whether $\alpha = 0.262$ is a universal property of the graph structure or an artifact of the weight distribution. We answer this with a systematic study across five distribution families (all at $V = 3{,}000$, $k \in \{4, 8, 16, 32, 64, 128\}$, 30 repetitions each; scripts `theory_probe_v4.py`, outputs in `resultsTheory/`).

Finding 1: α grows with weight range and shrinks with k-range. Two separate effects govern α:

- *Weight effect.* For $\mathrm{U}[1, W]$ at fixed $k \in [4, 64]$: α increases from 0.162 ($W = 10$) to 0.317 ($W \geq 10{,}000$), driven by the log-ratio $\log(W_{\max}/W_{\min})$ of available weight diversity. (Table II.)
- *k-range effect.* For continuous weights ($W \to \infty$), the fitted α is *not* constant but decreases as the fitted k-range expands: $\alpha = 0.317$ ($k \leq 64$), 0.285 ($k \leq 256$), 0.270 ($k \leq 512$). The power law is a finite-k approximation.

TABLE II

FITTED α FOR $\mathrm{U}[1, W]$ AT $k \in [4, 64]$ (PRACTICAL RANGE; $V = 3{,}000$, 30 REPS).

W	α	c	W	α	c
10	0.162	0.835	500	0.295	0.929
50	0.250	0.876	1,000	0.298	0.933
100	0.272	0.898	10^5	0.317	0.969
200	0.285	0.914	∞ (float)	0.317	0.969

Finding 2: α scales with $\log(W_{\max}/W_{\min})$. When the *range* is fixed at 100 but $W_{\min}$ varies, α tracks $\log(W_{\max}/W_{\min})$ rather than either endpoint: $W_{\min} \in \{1, 10, 100\}$ gives $\alpha \in \{0.272, 0.204, 0.096\}$. Intuitively, $\log(W_{\max}/W_{\min})$ measures how many "orders of magnitude" of weight diversity compete for shortest paths.

Finding 3: Bimodal (two-point) weights break the power-law form. For $\mathrm{TwoPoint}\{1, W_h\}$ with $P(W = 1) = P(W = W_h) = 0.5$, the power-law fit gives $R^2 < 0.52$ regardless of W_h: the weight-1 edges dominate BFS-like, leaving skip_ratio roughly constant in k ($\alpha \approx 0$).

Finding 4: Large-k convergence toward the harmonic bound. Extending to $k \in \{4, \ldots, 512\}$ reveals that the local slope $\alpha(k) \equiv -\partial \ln(1 - \text{skip\_ratio})/\partial \ln k$ decreases monotonically and *approaches* $1/H_k$ from above (see Figure 3). Concretely, for continuous weights:

$$\alpha(k{=}32{\to}64) \approx 0.270, \quad \alpha(k{=}128{\to}256) \approx 0.204,$$
$$\alpha(k{=}256{\to}512) \approx 0.183,$$

while $1/H_{64} \approx 0.211$, $1/H_{256} \approx 0.163$. The behavior of $\alpha(k)/(1/H_k)$ is *distribution-specific* and must be interpreted with care. For large-W and continuous-weight distributions ($\mathrm{U}[1, 10^5]$, $\mathrm{U}[0, 1]$ float), the ratio stabilizes around 1.18–1.19 for $k \geq 32$ and *approaches* 1.0 from above—this is the behavior illustrated in Figure 3. For the original $\mathrm{U}[1, 100]$ distribution used to calibrate the main formula, the ratio peaks near 1.0 at $k \approx 32$–64 and then *falls below* 1.0 for $k > 64$, reaching 0.68 at $k = 128$–256 and 0.51 at $k = 256$–512. This declining ratio explains the growing over-prediction of Equation (1) at $k \geq 128$ reported above: the true $\mathrm{U}[1, 100]$ skip_ratio curve is flatter than the fitted power law at high k, while the formula—calibrated for the full range $k \in [4, 512]$—makes the optimal average trade-off. In all cases, push_count$/V$ remains below H_k: the harmonic upper bound is never violated (ratio $= 0.855$ at $k = 512$ for large-W).

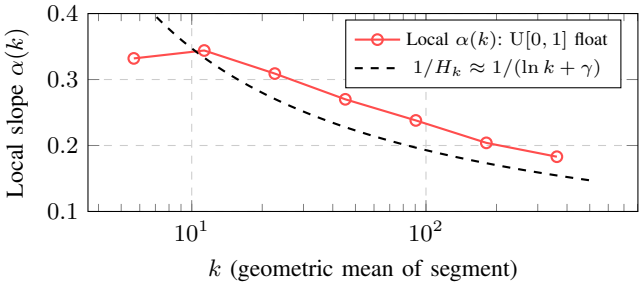

Fig. 3. Local slope $\alpha(k)$ converges toward $1/H_k$ as k grows, consistent with the asymptotic skip_ratio form $1 - c/H_k$ (theory_probe_v4, $V = 3{,}000$, continuous float weights).

Conjecture (two-regime model). The skip_ratio of lazy-deletion Dijkstra on locally tree-like graphs is consistent with a *two-regime* empirical model:

$$\text{skip\_ratio}(k) \approx \begin{cases} 1 - c(F, W) \,/\, k^{\alpha(F,W)}, & k \lesssim 100 \\ 1 - A(F) \,/\, H_k, & k \gg 100 \end{cases} \quad (2)$$

where F denotes the weight distribution, $\alpha(F, W) \in (0, 0.32]$ is strongly associated with $\log(W_{\max}/W_{\min})$, and $A(F) \to 1$ as $W \to \infty$ (the full i.i.d. harmonic limit). Three caveats apply. First, the crossover scale $k \approx 100$ is an *approximate heuristic*: the local slope decreases monotonically for all k rather than exhibiting a sharp regime boundary. Second, $A(F) \to 1$ is an asymptotic statement; at $k = 512$ we observe push_count$/V = 0.855\, H_{512}$, so the full harmonic limit lies beyond our experimental range. Third, bimodal distributions ($\alpha \approx 0$) are excluded; the power-law form does not apply when one weight dominates BFS-like. Formal derivation of the crossover scale and the functional form of $\alpha(F, W)$ remain open problems.

Practical implication. For the common engineering case of $k \leq 64$ with $\mathrm{U}[1, 100]$ weights, $\alpha \approx 0.27$–0.29 and Equation (1) is accurate to within ± 0.025 for $k \in [8, 64]$ in skip_ratio (max residual 0.022 at $k = 32$). For wider k ranges or other weight distributions, Table II provides interpolation guidance.

TABLE III
THRESHOLD HEURISTIC: ACCURACY VS. SPEEDUP (EXPERIMENT 5, $V = 1{,}000$, 5 RANDOM SEEDS). $\theta = \infty$ IS THE UNMODIFIED BASELINE.

Graph	θ	Wrong (%)	Push reduc.	Time (ms)	Speedup
sparse $k = 10$	∞	0.0	0%	3.69	1.00×
	3	5.3	6.7%	4.14	0.89×
	2	19.9	22.5%	3.38	1.09×
	1	53.3	52.9%	1.81	2.04×
dense-mid $k = 32$	∞	0.0	0%	11.07	1.00×
	3	17.9	20.3%	8.72	1.27×
	2	38.1	39.9%	6.54	1.69×
	1	69.0	66.8%	3.14	3.53×
dense-high $k = 64$	∞	0.0	0%	22.58	1.00×
	3	25.2	27.1%	14.17	1.59×
	2	49.9	46.9%	9.32	2.42×
	1	74.3	71.6%	4.67	4.83×

V. WHY NAÏVE FIXES FAIL

A natural attempt to reduce skip_ratio is to track how many times vertex v is currently in the heap (in_heap[v]) and block a push when in_heap[v] $\geq \theta$. We call this the *threshold heuristic* and evaluate it in Experiment 5.

Correctness catastrophe. As shown in Table III, $\theta = 1$—which allows at most one copy of each vertex in the heap—renders 53–74% of vertices with incorrect distances. The mechanism is straightforward: blocking a push creates a *propagation gap*: if vertex v is not re-pushed when its tentative distance improves, its neighbors will never learn of the improvement, cascading errors outward. This is not a bug in the heuristic; it is a structural property of lazy deletion. The "skip-on-pop" strategy *relies* on every relaxation being pushed so that the minimum always surfaces correctly.

No viable operating point. For sparse graphs ($k = 10$), $\theta = 3$ achieves only 0.89× speedup (slower than baseline) while producing 5.3% wrong nodes. For dense graphs, any $\theta \geq 2$ that yields noticeable speedup also corrupts more than 18% of answers. There is no threshold at which accuracy is acceptable and speedup is meaningful.

Insight. The failure of the threshold heuristic reveals that skip_ratio is not a symptom of *redundant* pushes—it is an inherent cost of the lazy deletion protocol. The correct remedy is not to reduce pushes but to choose a data structure whose operational semantics are compatible with the relaxation pattern.

VI. DATA STRUCTURE COMPARISON

A. Three-Way Comparison: Python and C++

We compare three algorithms under identical graph conditions (Experiments 6 and 7):

- **heapq**: lazy deletion with Python's `heapq` / C++ `priority_queue<>`.
- **Fibonacci**: decrease-key Dijkstra with a hand-crafted Fibonacci heap (Python and C++, same from-scratch implementation). Correctness verified by comparing distances against heapq output on all tested instances; maximum distance error is 0 throughout.

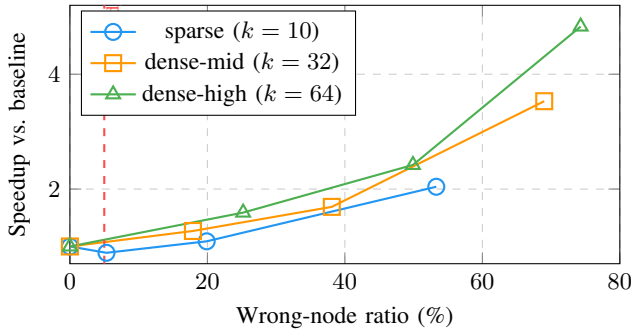

Fig. 4. Threshold heuristic trade-off: wrong-node ratio vs. speedup (Experiment 5, $V = 1{,}000$). Every point on each curve corresponds to a different θ (∞, 3, 2, 1 from left to right). No operating point meets a 5% error budget with meaningful speedup.

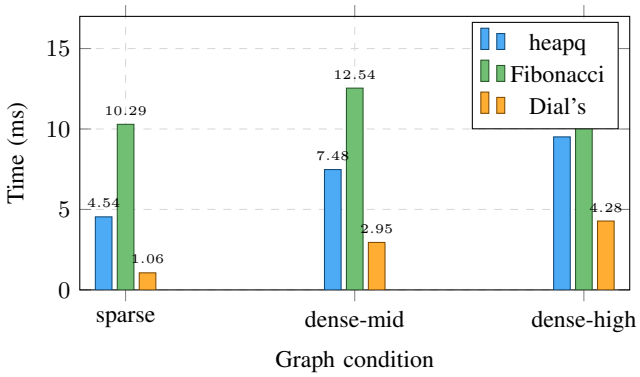

Fig. 5. C++ wall-clock time for $V = 10{,}000$ (Experiment 7). Dial's dominates across all densities; Fibonacci is consistently slowest.

TABLE IV
WALL-CLOCK TIME (MS) FOR $V = 10{,}000$, 10 RUNS, C++ (EXPERIMENT 7). ALL RESULTS VERIFIED CORRECT ($\max\_err = 0$).

Graph condition	heapq	Fibonacci	Dial's
sparse ($k = 8$)	4.54	10.29	**1.06**
dense-mid ($k = 32$)	7.48	12.54	**2.95**
dense-high ($k = 64$)	9.51	14.29	**4.28**
Fibonacci / heapq ratio			
sparse	2.27× slower		
dense-mid	1.68× slower		
dense-high	1.50× slower		
Dial's speedup over heapq			
sparse	4.28× faster		
dense-mid	2.53× faster		
dense-high	2.22× faster		

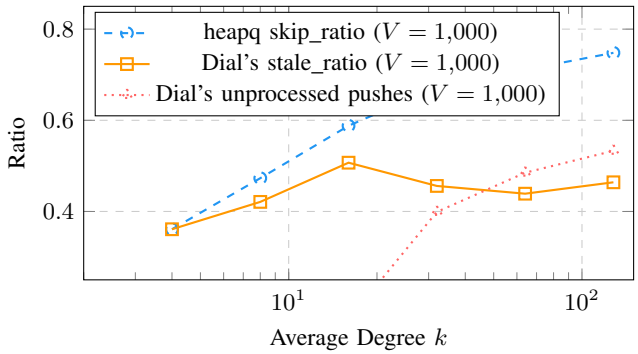

Fig. 6. heapq skip_ratio vs. Dial's stale_ratio and unprocessed-push fraction as functions of k (Experiment 8, $V = 1{,}000$, log x-axis). As k grows, heapq's skip_ratio climbs monotonically while Dial's stale_ratio plateaus; the gap is absorbed by unprocessed pushes.

- **Dial's**: circular bucket queue, integer weights $W = 100$.

Table IV reports wall-clock times for $V = 10{,}000$ in C++. Figure 5 shows the corresponding bar chart.

Dial's speedup: peak vs. mean. Dial's delivers a **peak** speedup of $4.28\times$ over heapq in the sparse C++ setting ($k = 8$, $V = 10{,}000$). Across all three density conditions ($k \in \{8, 32, 64\}$), the geometric mean speedup is $\mathbf{2.9}\times$ in C++ and $\mathbf{1.5}\times$ in Python. The Python gap is narrower because the Python heap-push overhead (Python object allocation) partially offsets the architectural advantage of Dial's circular array. Throughout the paper we report the C++ figures as the language-independent performance bound and note Python figures where relevant.

Fibonacci heap: language-independent underperformance. In Python (Experiment 6), Fibonacci is $2.35\times$ slower than heapq for $k = 8$, $V = 10{,}000$ (157 ms vs. 67 ms). The C++ re-implementation (Experiment 7) yields $2.27\times$ (10.3 ms vs. 4.5 ms)—essentially the same ratio, confirming that the gap is *not* a Python artifact. The Python/C++ speedup is $14.8\times$ for heapq and $15.3\times$ for Fibonacci, demonstrating that the language overhead is proportional for both algorithms. The Fibonacci underperformance therefore stems from pointer-chasing and cache-unfriendly consolidation, not from interpreter overhead.

B. Dial's Natural Pruning Mechanism

To understand why Dial's achieves up to $4.28\times$ speedup, we instrument its bucket operations in Experiment 8.

Unprocessed pushes. Let P denote total bucket pushes and S denote stale skips. In a lazy deletion heap, $P = V + S$ (conservation), so every pushed entry is either settled or skipped. In Dial's, however, $P > V + S$: pushes to future buckets beyond the maximum settled distance are *never popped at all*. We call these *unprocessed pushes* and denote their fraction $\rho_{\text{unp}} = (P - V - S)/P$.

Figure 6 tells the story. For $k = 64$, heapq skip_ratio is 0.711 and Dial's stale_ratio is only 0.439—a difference of 0.272. That difference is accounted for by unprocessed pushes ($\rho_{\text{unp}} = 0.485$): nearly **49% of all Dial's pushes are never processed**, because the algorithm terminates once all V nodes are settled, leaving future-bucket entries abandoned. By contrast, the binary heap must pop every pushed entry.

The mechanism has an elegant explanation: in Dial's, settling node u at distance d pushes its neighbors into buckets at $d + w$, some of which may be far ahead of the current bucket pointer. When the nearer of u's neighbors are settled later via better paths, those far-future entries become both stale

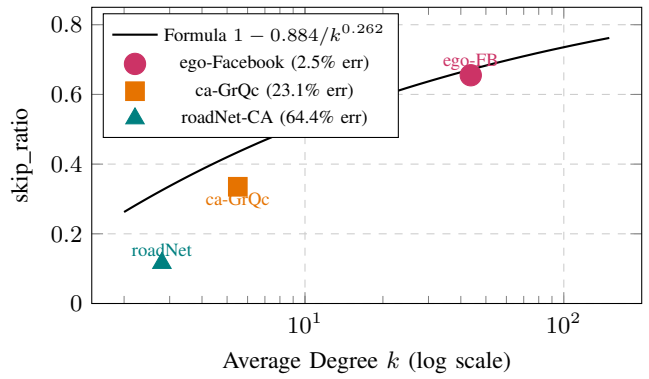

Fig. 7. skip_ratio formula validated on SNAP datasets (Experiment 10). Black curve: formula (1) fitted on random graphs. Colored markers: measured skip_ratio on real graphs.

and unreachable before the pointer reaches their bucket. Dense graphs (k large) produce more such "orphaned" entries. This is a qualitative advantage that binary heaps cannot replicate.

C. Integer vs. Float Weights

Experiment 9 repeats the Python and C++ comparison with float weights $w \sim \mathcal{U}[0,1]$ (excluding Dial's, which requires integer weights). The relative ordering is unchanged: C++ Fibonacci is 1.6–2.7× slower than heapq at $V = 10{,}000$, and the Python/C++ ratio is 15× for both algorithms. We conclude that the performance relationship between heapq and Fibonacci heap is **independent of weight type**, generalizing our findings beyond integer graphs.

D. Data-Structure Selection Framework

Effect of maximum edge weight W (Experiment B). All prior comparisons used W=100. Experiment B varies $W \in \{100, 500, 1{,}000, 5{,}000, 10{,}000\}$ with V=10,000 fixed to characterize the interaction between W and Dial's speedup. For sparse graphs (k=8), Dial's advantage erodes sharply: speedup is 2.85× at W=1,000 but falls to 0.80× at W=5,000 and 0.66× at W=10,000, placing the breakeven threshold W^* in the range [1,000, 5,000] for this setting (the measured points bracket the crossover; precise location requires finer W sampling). For denser graphs ($k \geq 32$), Dial's remains faster than heapq even at W=10,000 (≥1.07× speedup), so $W^* > 10{,}000$ in this regime. The mechanism is the circular-array traversal cost: with $W+1$ buckets, the number of bucket-scan iterations grows proportionally to W, eventually dominating the $O(E \log E)$ savings.

Table V consolidates our findings into actionable guidance. The decision depends on two observable graph properties: weight type and average degree k.

VII. Real-World Validation

We test formula (1) on three SNAP [12] datasets. Integer weights $w \sim \mathcal{U}_{\mathbb{Z}}[1, 100]$ are assigned randomly (seed 42); skip_ratio is averaged over 10 random source vertices. Results appear in Table VI and Figure 7.

TABLE V
Data-structure selection framework. $k_{\text{low}} \approx 10$ separates regimes where skip_ratio < 0.5. W^* is the maximum edge weight below which Dial's beats heapq (Experiment B; V=10,000).

Condition	Recomm.	Rationale
Integer weights, any k, $W \leq W^*$ †	Dial's bucket queue	Up to 4× speedup (sparse) via natural pruning; O(V+E+W).
Integer weights, k=8, $W \gtrsim 5{,}000$	heapq	Dial's speedup < 1 at W=5,000 (0.80×) and 0.66× at W=10,000; circular-array traversal dominates.
Float weights, $k \leq 10$ (sparse)	heapq (lazy deletion)	skip_ratio < 0.5; overhead acceptable; simplest implementation.
Float weights, $k > 10$, C++ (supplemental)	Consider indexed/4-ary	Supplemental evidence (§VIII): indexed 1.4–2.2× faster, 4-ary 1.4–1.6× faster; verify on target platform.
Float weights, $k > 10$, Python	heapq (lazy deletion)	Indexed and d-ary heaps slower than heapq in Python; lazy wins.
Fibonacci heap	Avoid in practice	1.5–2.3× slower than heapq in C++ due to pointer-chasing; theoretical O($V \log V$+E) advantage is not realized.

† W^* depends on k: for k=8, $W^* \in$ [1,000, 5,000] (crossover observed between these measured values); for $k \geq 32$, Dial's remains faster even at W=10,000 (≥1.07× speedup), so $W^* > 10{,}000$ in this regime. See Section VI-D.

TABLE VI
Formula validation on SNAP real-world graphs (Experiment 10).

Dataset	V	E	k	Measured skip_ratio	Predicted (Eq. 1)
ego-Facebook	4,039	88,234	43.7	0.655	0.671
ca-GrQc	5,241	14,484	5.5	0.335	0.435
roadNet-CA	1,965,206	2,766,607	2.8	0.116	0.326

	Relative error
ego-Facebook	**2.5%**
ca-GrQc	23.1%
roadNet-CA	64.4%

ego-Facebook (2.5% error). This dense social network (k = 43.7) closely resembles an Erdős–Rényi graph: high average degree, low clustering relative to k, and short cycles of length ≥ 4. The locally tree-like assumption holds approximately, and the formula achieves 2.5% relative error—well within experimental noise.

ca-GrQc (23.1% error). The Arxiv collaboration network has $k = 5.5$, placing it at the left tail of the power-law curve. The error has two separable components. *Small-k component:* even on an ER random graph at $k \approx 5.5$, the formula carries $\Delta \approx 2$–4% inaccuracy (by interpolation from the $k = 4$ under-prediction of $\Delta \approx 0.03$); this accounts for roughly 2–4 percentage points of the 23% gap. *Structural component:* collaboration networks exhibit high clustering (many short cycles), violating the tree-like assumption on which the formula rests; the residual ≈ 19–21% error is attributable to this structural mismatch. The two effects are therefore roughly 1:7 in magnitude.

roadNet-CA (64.4% error). This is not a limitation of the formula but is *consistent with the scope of its derivation*. Road networks are planar graphs (bounded genus, no long-range edges); their average degree $k = 2.8$ is far below the range for which the locally tree-like assumption holds, and their structure introduces long-range correlations in settling order. Measured skip_ratio is 0.116 vs. predicted 0.326—road networks exhibit markedly *less* heap waste than random graphs of the same degree because most paths are monotone along road corridors. Applying formula (1) to planar graphs would be a misuse; we treat the deviation as confirmation that the formula's applicability boundary is correctly identified by the locally tree-like condition.

Scale-free (Barabási–Albert) graphs (Experiment C). Real social and technological networks often follow a power-law degree distribution rather than the Poisson distribution of Erdős–Rényi graphs. We test formula (1) on Barabási–Albert (BA) graphs [10] with attachment parameter $m \in \{4, 8, 16, 32\}$ (expected $k_{\text{avg}} = 2m \in \{8, 16, 32, 64\}$), $V \in \{1{,}000,\ 10{,}000\}$, random integer weights, and 10 repetitions per configuration.

The formula generalizes well to scale-free structure: at $k_{\text{avg}} \geq 16$, relative error is 2–4% for both BA and ER graphs—essentially indistinguishable. At $k_{\text{avg}} \approx 8$ (sparse), BA graphs show a modestly lower skip_ratio (≈ 0.450) than ER graphs with matching k_{avg} (≈ 0.487), yielding 7–8% relative error instead of $< 6\%$. The gap is attributable to degree heterogeneity in BA graphs: hub nodes (high-degree) are settled early and generate fewer subsequent relaxations per unit of k_{avg}, slightly reducing the stale-pop fraction. Nonetheless, the formula provides a useful first approximation for scale-free graphs, extending its applicability beyond Erdős–Rényi random graphs.

VIII. Discussion and Limitations

Scope of validity. Formula (1) is calibrated on Erdős–Rényi-like random graphs with uniform edge weights. It generalizes to dense social networks (ego-Facebook, 2.5% error) but not to planar or hierarchical graphs. For structured inputs, the skip_ratio can be substantially lower than the formula predicts, as roadNet-CA demonstrates. A practitioner should verify that their target graph exhibits high average degree and low local clustering before applying the formula.

Fibonacci heap: theoretical vs. practical optimality. The Fibonacci heap achieves the theoretically optimal $O(V \log V + E)$ bound, yet our C++ measurements show it is 1.5–2.3× slower than a binary heap for $V \leq 10{,}000$ across all tested densities. This gap is language-independent—we replicate it in both Python and C++ with the same ratio—and stems from pointer-chasing overheads during consolidation and cascading cuts. For graphs where $E \gg V \log V$ (i.e., extremely dense), the Fibonacci heap's asymptotic advantage might eventually manifest, but such densities exceed the range studied here. We recommend against Fibonacci heaps unless theoretical guarantees are mandated by a formal proof system.

Indexed and d-ary heaps (supplemental experiment). To probe the two most commonly suggested alternatives to lazy binary heapq, we ran a Python supplemental experiment (`dijkstra_heap_compare.py`) across $V \in \{500, 1000, 3000, 5000, 10000\}$ and $k \in \{4, 8, 16, 32, 64, 128\}$ with five repetitions each ($\text{U}[1, 100]$). *Hardware note:* this experiment ran on a local MacBook Pro (Apple M3, macOS), not on the GCP e2-standard-4 instance used for the main experiments; timing comparisons within this experiment are internally consistent, but cross-experiment absolute times should not be compared directly.

Skip-ratio is implementation-invariant. All three lazy-deletion variants (binary, 4-ary, 8-ary heaps) produced skip_ratios within ± 0.001 of each other at every (V, k) tested. Representative values at $V = 3{,}000$: 0.242, 0.438, 0.573, 0.660, 0.711, 0.746 for $k = 4, 8, 16, 32, 64, 128$ respectively. As expected, the *indexed* binary heap achieves skip_ratio $= 0$ throughout by maintaining a position array for $O(\log V)$ decrease-key.

Lazy binary heapq vs. indexed heap (Python). Despite zero skip overhead, the indexed heap is consistently *slower* than lazy heapq for $k \leq 32$: speedup (indexed/lazy) ranges from $3.4\times$ at $k = 4$ to $1.5\times$ at $k = 32$ ($V = 10{,}000$), because each decrease-key requires a random-access position lookup and upward sift, whereas heapq's lazy push is a simple append. The gap closes near $k \approx 64$–128 (speedup ≈ 1.0), where the high skip_ratio (≈ 70–75%) begins to offset indexed heap's bookkeeping cost. These Python results corroborate the $1.5\times$ geometric-mean speedup of lazy heapq reported in Section VI.

d-ary heaps (Python). 4-ary and 8-ary lazy heaps are uniformly *slower* than binary heapq across all (V, k) configurations tested. The extra branching factor reduces tree height but adds per-level comparison work with no cache benefit in Python's interpreted loop; the slowdown versus binary ranges from $1.2\times$ at $k = 4$ to $1.7\times$ at $k = 64$.

C++ heap comparison. We repeated the four-variant comparison in `dijkstra_heap_compare.cpp` (`-O2`, Apple M3, directed ER, $m = Vk$ edges). The findings reverse relative to Python. First, in C++ the 4-ary lazy heap is consistently **faster** than the lazy binary heap by 1.4–1.6× ($V = 10{,}000$: 0.76 ms vs. 1.22 ms at $k = 4$; 3.95 ms vs. 5.61 ms at $k = 128$)—the reduced tree height offsets the wider sift-down scan once the interpreted-loop overhead is removed. Second, the indexed binary heap outperforms lazy binary by

1.4–2.2× for $k \geq 8$ ($V = 10{,}000$, $k = 32$: 1.31 ms vs. 2.83 ms, 2.2×); this advantage grows with k because the lazy heap accumulates $O(\text{push\_count})$ entries while the indexed heap maintains exactly V entries throughout. In Python the reverse held: heapq's C backend makes each push cheaper than an interpreted decrease-key lookup. The C++ results suggest that for high-k production code an indexed heap or 4-ary lazy heap ($d = 4$) is preferable to lazy binary deletion, even absent a skip-ratio reduction (full data: `resultsHeapCompare/`). *Pairing heaps* offer favorable amortized decrease-key and remain uncharacterized under lazy deletion—an open extension.

Statistical note. Main experiments (Sections IV–VI) use 5–10 repetitions per (V, k) configuration and report means. The observed inter-run variation is ≤ 0.013 in skip_ratio across all (V, k) pairs (Table I), indicating high stability. Weight-distribution experiments (Section IV-D) use 30 repetitions ($V = 3{,}000$).

Source-vertex sensitivity. All main experiments use source vertex 0 (seed 42). To verify that this design choice does not inflate or suppress the measured skip_ratio, we ran a supplementary experiment (`source_sensitivity.py`) sampling 20 uniformly random source vertices across 5 graph instances, for $V \in \{1{,}000, 3{,}000, 10{,}000\}$ and $k \in \{4, 8, 16, 32, 64, 128\}$ (U[1, 100]). The graph model matches `theory_probe_v4.py` exactly: undirected ER with $m = \lfloor Vk/2 \rfloor$ random edges plus a random spanning tree, so skip_ratio values are directly comparable to the main results. The coefficient of variation of skip_ratio across sources is at most 2.5% (at $V = 1{,}000$, $k = 4$) and falls below 0.5% for $k \geq 64$. Standard deviation across sources does not exceed 0.010 at any (V, k) tested. We conclude that skip_ratio is effectively source-independent for the ER-like graphs studied here, consistent with the locally tree-like theory (the expected push count per vertex depends only on local degree structure, not on which vertex is the source).

Future work. Three open questions arise naturally from this study:

1) *Theoretical derivation of the two-regime empirical model.* Section IV-D establishes empirically that the local slope $\alpha(k)$ converges toward $1/H_k$ and that the push_count$/V$ ratio approaches H_k from below as $k \to \infty$. A formal proof connecting the record-minimum process on Poisson–Galton–Watson trees to both the power-law regime and the harmonic limit—and a derivation of the crossover scale $k \approx 100$—remains open.
2) *Exact characterization of* $\alpha(F, W)$. The dependence $\alpha \approx f(\log(W_{\max}/W_{\min}))$ is empirically clear but analytically unproven. An exact formula for the functional $\alpha(F, W)$ and its limiting value under continuous distributions would close this gap.
3) *Structured-graph models.* The 64% error on roadNet-CA motivates a separate characterization of skip_ratio for planar and hierarchical graphs, which have known algebraic structure amenable to analytic approaches.

IX. Conclusion

We have presented a systematic empirical characterization of the lazy deletion overhead in Dijkstra's algorithm as a closed-form *empirical model* of graph structure, filling a gap in prior experimental surveys that treat this overhead as a fixed implementation detail. Our key findings are:

1) **Power-law skip_ratio, V-independent.** skip_ratio$(k) \approx 1 - 0.884/k^{0.262}$ ($R^2 = 0.984$) holds across graph sizes from 500 to 10^6 vertices, confirming that skip_ratio is primarily determined by average degree k under fixed weight distribution and ER-like graph topology. The formula generalizes to Barabási–Albert scale-free graphs with $< 8\%$ relative error (Experiment C).
2) **Two-regime empirical model and weight-distribution dependence.** The exponent α is not universal: it is strongly associated with $\log(W_{\max}/W_{\min})$ and ranges from 0 (constant weights) to ≈0.32 (continuous uniform, $k \leq 64$). At larger k, the local slope converges toward $1/H_k$, revealing that the power-law regime transitions to a harmonic regime $1 - A/H_k$ as $k \to \infty$—consistent with the classical i.i.d. record-minimum bound and never violating it.
3) **Threshold fixes are broken.** Push-blocking heuristics cause propagation gaps corrupting up to 74% of shortest distances at any setting that yields measurable speedup.
4) **Fibonacci heap is not practical.** Despite $O(V \log V + E)$, Fibonacci heap is 1.5–2.3× slower than heapq in optimized C++, language- and density-independent.
5) **Dial's natural pruning delivers 4× peak speedup.** Up to 49% of Dial's bucket pushes are never processed—a structural property with no binary-heap analog—enabling **peak** 4.3× speedup for sparse integer-weight graphs and a geometric mean of 2.9× in C++ (1.5× in Python).

The practical takeaway is: *skip_ratio is a tight function of average degree k and weight distribution; matching graph structure to the right priority queue via our selection framework achieves up to* 4.3× *peak speedup.* Specifically: use Dial's for bounded integer weights ($W \lesssim 1{,}000$ for k=8; any W for $k \geq 32$); use heapq as the portable float-weight baseline; consider indexed or 4-ary heaps for optimized C++ implementations at higher k (supplemental evidence, §VIII); and avoid Fibonacci heaps in practice.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
[2] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
[3] R. B. Dial, "Algorithm 360: Shortest-path forest with topological ordering," *Communications of the ACM*, vol. 12, no. 11, pp. 632–633, 1969.
[4] A. V. Goldberg, H. Kaplan, and R. F. Werneck, "Better landmarks within reach," in *Proc. Workshop on Experimental Algorithms (WEA)*, 2007, pp. 38–51.
[5] P. Sanders and D. Schultes, "Highway hierarchies hasten exact shortest path queries," in *Proc. ESA*, 2005, pp. 568–579.

[6] M. Chen, E. Felsner, and A. Schulz, "On the performance of priority queues for shortest path algorithms," in *Proc. ALENEX*, 2007, pp. 52–63.
[7] B. V. Cherkassky, A. V. Goldberg, and T. Radzik, "Shortest paths algorithms: Theory and experimental evaluation," *Mathematical Programming*, vol. 73, pp. 129–174, 1996.
[8] D. H. Larkin, S. Sen, and R. E. Tarjan, "A back-to-basics empirical study of priority queues," in *Proc. ALENEX*, 2014, pp. 61–72.
[9] B. Bollobás, *Random Graphs*, 2nd ed. Cambridge University Press, 2001.
[10] A.-L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, pp. 509–512, 1999.
[11] P. Flajolet and R. Sedgewick, *Analytic Combinatorics*. Cambridge University Press, 2009.
[12] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford Large Network Dataset Collection," http://snap.stanford.edu/data, Jun. 2014.
[13] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
[14] C. Demetrescu, A. V. Goldberg, and D. S. Johnson, Eds., *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, vol. 74, DIMACS Series in Discrete Mathematics and Theoretical Computer Science. American Mathematical Society, 2009.